

C 语言程序设计

第十四讲 位运算



张 华

主要内容

- 位运算的概念
- 位运算及其应用
- 位运算复合赋值运算
- 位段
- 位段应用实例

位运算的概念

■ 位（bit）是数据存储的最小单位。

✱ 在计算机中的二进制数系统中，每个0或1就是一个位。

■ 位运算是指按二进制位进行的运算。

■ 在系统软件中，常要处理二进制位的问题。

■ C语言提供位运算的功能，与其他高级语言（如Basic）相比，具有很大的优越性。

位运算符

位运算符表

运算符	含义	运算符	含义
&	按位与	~	取反
	按位或	<<	左移
^	按位异或	>>	右移

说明:

(1)运算量只能是整型或字符型的数据。

(2)运算符中除位反(~)外, 均为二目运算符, 即要求两侧各有一个运算量。

位运算：&

&（按位“与”）

✿ 运算规则：

$0 \& 0 = 0$, $0 \& 1 = 0$, $1 \& 0 = 0$, $1 \& 1 = 1$

➤ 即只有当两者都为真时才为真

✿ 例如， $13 \& 7$ 的值为5，运算过程如下：

$$\begin{array}{r} 13 = 00001101 \\ (\&) 7 = 00000111 \\ \hline 00000101 \end{array}$$

按位与的用途

(1) 清零

- ✿ 若想对一个存储单元清零，即使其全部二进制位为 0，只要找一个二进制数，其中各个位符合以下条件：原来的数中为 1 的位，新数中相应位为 0。
- ✿ 然后使二者进行 & 运算，即可达到清零目的。

按位与的用途

（2）取一个数中某些指定位

- 如有一个整数 **a**（2字节），想要取其中的低字节，只需将 **a** 与 8 个 1 按位与即可。

a	0 0 1 0 1 1 0 0	1 0 1 0 1 1 0 0
b	0 0 0 0 0 0 0 0	1 1 1 1 1 1 1 1
c	0 0 0 0 0 0 0 0	1 0 1 0 1 1 0 0

按位与的用途

(3) 保留一位

- ✱ 与一个数进行 & 运算，此数在该位取 1。
- ✱ 例：有一数 0 1 0 1 0 1 0 0，想把其中第 3、4、5、7、8 位保留下来，运算如下：

```
01010100(84)
&00111011(59)
-----
00010000(16)
```

即： $a = 84$, $b = 59$
 $c = a \& b = 16$

位运算：

| (按位“或”)

✱ 运算规则为：

$0|0=0$, $0|1=1$, $1|0=1$, $1|1=1$

➤ 即只要有一个为真其结果为真

✱ 例如， $5|9$ 的值为13，运算过程如下：

$5 = 00000101$

$(|) 9 = 00001001$

00001101

按位或的应用

按位或运算常用来对一个数据的某些位定值为 1。

✿ 例如：如果想使一个数 a 的低 4 位改为 1，只需将 a 与 00001111（15）进行按位或运算即可。

✿ 例如： a 是一个整数（16 位），
有表达式： $a | 0xFF$
则低 8 位全置为 1，高 8 位保留原样。

位运算：^

^ (按位“异或”)

✱ 运算规则为：

$0 \wedge 0 = 0$, $0 \wedge 1 = 1$, $1 \wedge 0 = 1$, $1 \wedge 1 = 0$

➤ 即当两者取值相异时为真

✱ 例如， $10 \wedge 5$ 的值为15，运算过程如下：

$$\begin{array}{r} 10 = 00001010 \\ (\wedge) 5 = 00000101 \\ \hline 00001111 \end{array}$$

按位异或的应用

(1) 使特定位翻转

✿ 设有 0 1 1 1 1 0 1 0，想使其低 4 位翻转，即 1 变为 0，0 变为 1。可以将它与 0 0 0 0 1 1 1 1 进行 ^ 运算，即：

运算结果的低 4 位正好是原数低 4 位的翻转。可见，要使哪几位翻转就将其进行 ^ 运算的该几位置为 1 即可。



	01111010
^	00001111
	01110101

按位异或的应用

（2）与0相 $\wedge$ ，保留原值

✱ 例如：012 $\wedge$ 00=012

$$\begin{array}{r} 00001010 \\ \wedge 00000000 \\ \hline 00001010 \end{array}$$

因为原数中的1与0进行 $\wedge$ 运算得1，0 $\wedge$ 0得0，故保留原数。

位运算：~

~ (按位“取反”)

✱ 运算规则为：

~0=1, ~1=0

➤ 取相反的值

✱ 例如，~7的值为-8，运算过程如下：

$(\sim) 7 = 00000111$

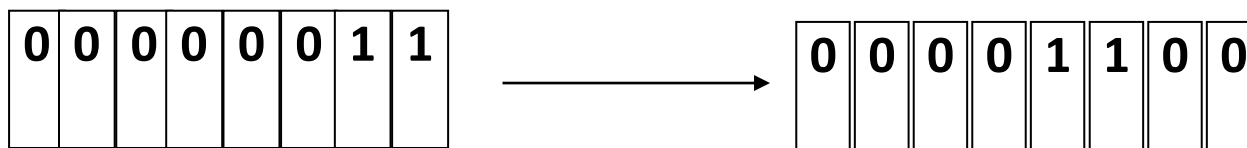
11111000

位运算：<<

<< (“左移”)

- 用来将一个数的各二进制位全部左移若干位，右边空缺位补0。
- 例如，将3左移2位，结果为12。

左移2位



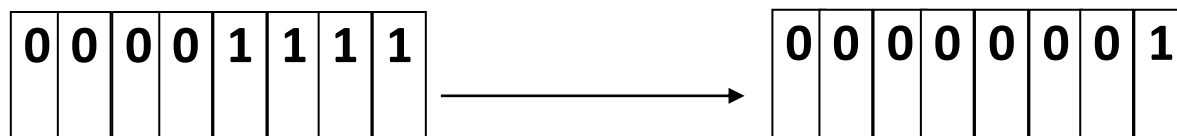
左移1位相当于该数乘以2，左移n位相当于该数乘以 2^n 。因此，将3左移2位，相当于3乘以4。

位运算：>>

>> (“右移”)

- 用来将一个数的各二进制位全部右移若干位。移去右端的位被舍弃，若为无符号数，左端补0。例如，将15右移3位，结果为1。

右移3位



右移1位相当于该数除以2，右移n位相当于该数除以 2^n ，因此，将15右移3位，相当于 $15/2^3 = 1$ (C语言规定整数相除商为整数)。

位运算: >>

右移时应注意符号问题。

- 对于无符号数，右移时左端补0。
- 对于有符号数，若符号位为0（该数为正），则右移时左端补0，同无符号数的处理。若符号位为1（该数为负），则右移时左端是补0还是补1，取决于所用的计算机系统。有的系统左端补0，称逻辑右移；左端补1，称算术右移。显然，两种方式所得的结果是不一样的。

例如：

a: 1111111111101111（补码，其十进制数是-17）

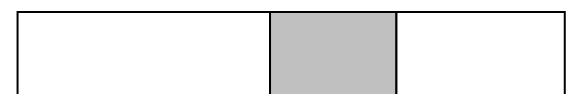
a>>3 0001111111111101（逻辑右移，高位补0）

a>>3 1111111111111101（算术右移，其十进制数是-3）

程序举例

例：取一个无符号数的4 ~ 7位。

（1）右移4位，使要取出的位移到最右端。



15 7 4 0



15 3 0

程序举例

 **例：取一个无符号数的4 ~ 7位。**

✿ (2) 根据提取指定位的方法，和00000000000001111（十六进制数0x000F）进行位与运算，程序如下：

```
#include <stdio.h>
int main(void)
{
    unsigned a , b ;
    scanf("%d", &a);
    b = a >> 4;
    b = b & 0x000F;
    printf("\na=%d b=%d", a, b);

    return 0;
}
```

115

a=115 b=7

位运算赋值运算符

位运算符与赋值运算符可以组成复合赋值运算符。

✱ $\&=$, $|=$, $>>=$, $<<=$, $\wedge=$

例: $a \&= b$ 相当于 $a = a \& b$

$a \<\<= 2$ 相当于 $a = a \<\< 2$

位段

■ 数据的存取一般以字节为单位。

✿ 实际上，有时存储一个数据不必用一个或多个字节。

✿ 例如，“真”或“假”用 0 或 1 表示，只需 1 位即可。

■ 在某些应用中，如过程控制、参数检测或数据通信等领域，特别是对硬件端口的操作，控制信息往往只占一个字节中的一个或几个二进制位来标志某些端口的状态或特征。而这些状态或特征只需要一个机器字中的一位或连续若干位来表示。

位段

■ 怎样向一个字节中的一个或几个二进制位赋值和改变它的值呢？

■ 可以用两种方法：

✱ (1) 可以人为地将一个整型变量分为几部分。

➤ 但是用这种方法给一个字节中某几位赋值太麻烦。

✱ (2) 位段

➤ C语言允许在一个结构体中以位为单位来指定其成员所占内存长度，这种以位为单位的成员称为“位段”或称“位域” (bit field)。

➤ 利用位段能够用较少的位数存储数据。

位段

位段结构中位段的定义格式为：

✿ **unsigned <成员名>: <二进制位数>**

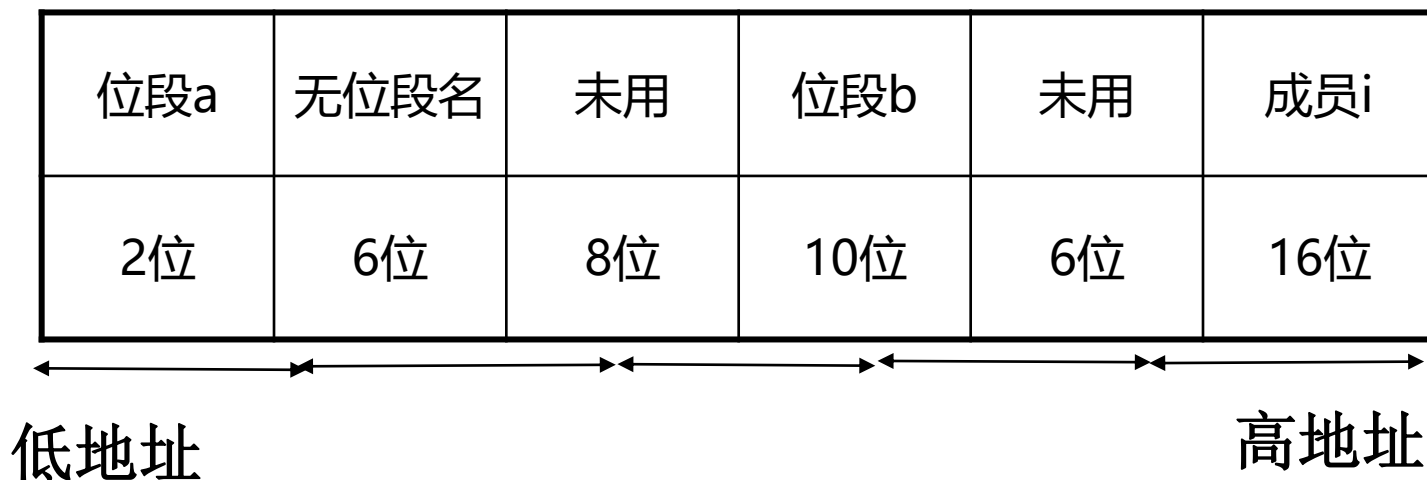
✿ 例如：

```
struct bytedata
{
    unsigned a: 2;    /*位段a, 占2位*/
    unsigned : 6;     /*无名位段, 占6位, 但不能访问*/
    unsigned : 0;     /*无名位段, 占0位, 表示下一位段从下一字边界开始*/
    unsigned b: 10;   /*位段b, 占10位*/
    int i;    /*成员i, 从下一字边界开始*/
} data;
```

位段

位段结构举例

对16位的编译系统而言，data变量的内存分配示意图如下。

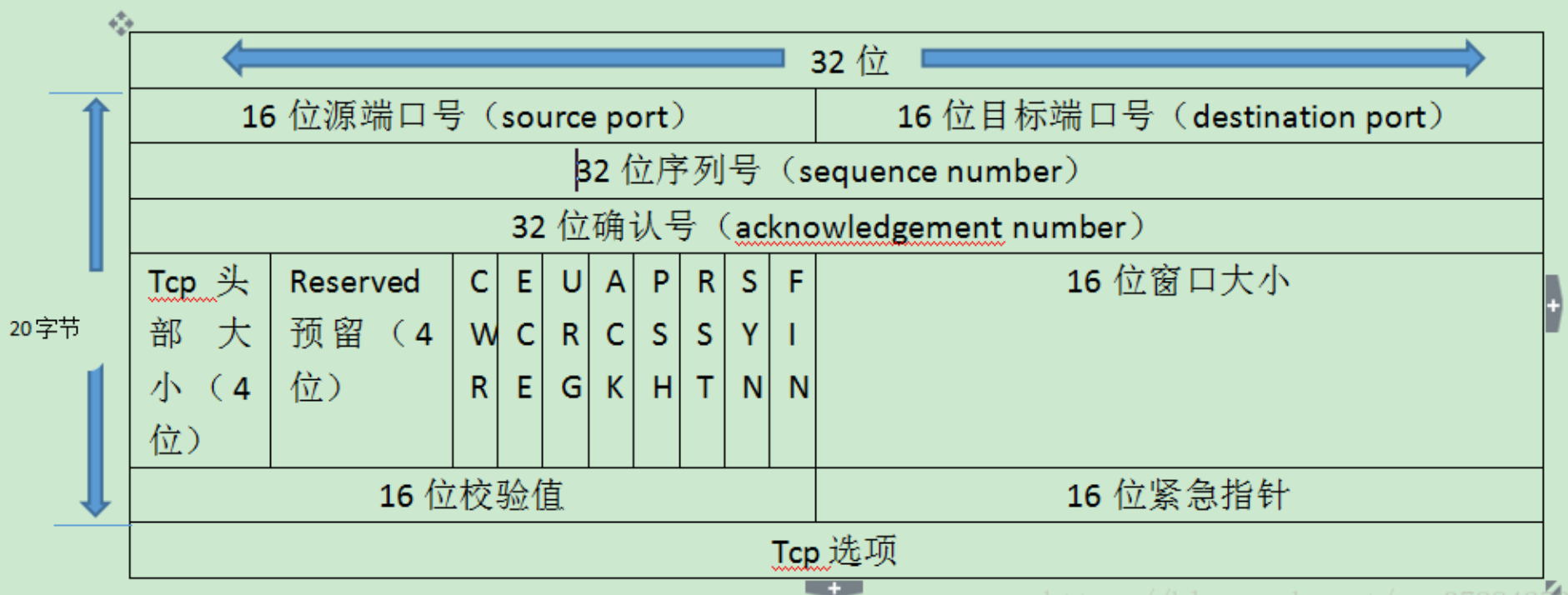


data变量的内存分配示意图

注：位段成员的类型必须指定为整数类型

位段应用举例

TCP消息头



位段应用举例

TCP消息头

```
struct TCPHeader {  
    short SrcPort; // 16位源端口号  
    short DstPort; // 16位目的端口号  
    int SerialNo; // 32位序列号  
    int AckNo; // 32位确认号  
    unsigned char HaderLen : 4; // 4位首部长度  
    unsigned char : 0; // 保留6位中的4位  
    unsigned char Reserved : 2; // 保留6位中的2位  
    unsigned char URG : 1;  
    unsigned char ACK : 1;  
    unsigned char PSH : 1;  
    unsigned char RST : 1;  
    unsigned char SYN : 1;  
    unsigned char FIN : 1;  
    short WindowSize; // 16位窗口大小  
    short TcpChkSum; // 16位TCP检验和  
    short UrgentPointer; // 16位紧急指针  
};
```

位段

关于位段数据，注意以下几点：

- ✿ 一个位段必须存储在同一存储单元（即字）之中，不能跨两个单元。如果其单元空间不够，则剩余空间不用，从下一个单元起存放该位段。
- ✿ 可以通过定义长度为0的位段的方式使下一位段从下一存储单元开始。
- ✿ 可以定义无名位段。
- ✿ 位段的长度不能大于存储单元的长度。
- ✿ 位段无地址，不能对位段进行取地址运算。
- ✿ 位段可以以%d，%o，%x格式输出。
- ✿ 位段若出现在表达式中，将被系统自动转换成整数。

小结

- 按二进制位进行运算，是C语言的特点之一。
- 通过位运算可以实现特殊应用。
- 在结构体中可以定义以位为单位的成员，即位段，充分利用存储空间。